

분산분석 예제

```
In [3]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# 정규분포, t분포, 신뢰구간(z분포, t분포), 가설검정(z-test, t-test, paired t-test)
import scipy.stats as stats

import warnings
warnings.filterwarnings(action = 'ignore')
```

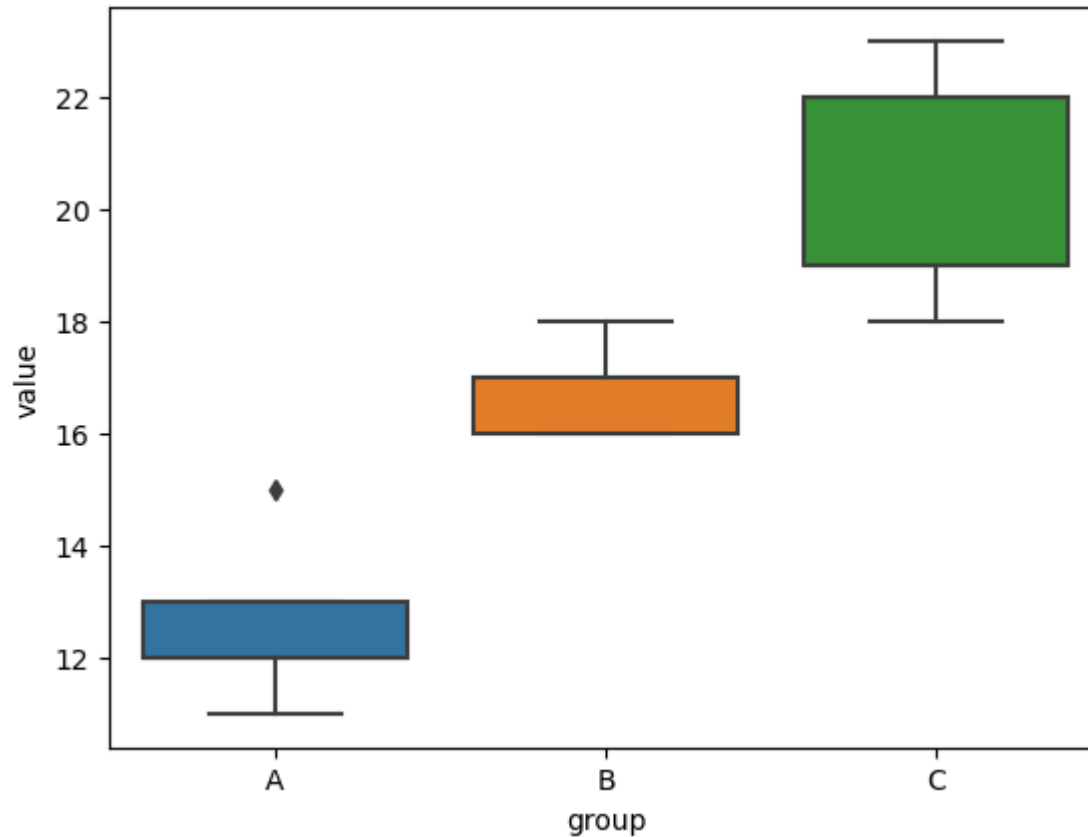
Oneway ANOVA

```
In [3]: data1 = pd.read_csv("headlight.csv")
data1
```

Out[3]:

	value	group
0	15	A
1	11	A
2	12	A
3	13	A
4	12	A
5	18	B
6	17	B
7	16	B
8	17	B
9	16	B
10	22	C
11	23	C
12	19	C
13	18	C
14	19	C

In [4]: `sns.boxplot(data=data1, x="group", y="value")`Out[4]: `<Axes: xlabel='group', ylabel='value'>`



```
In [5]: from statsmodels.formula.api import ols
        from statsmodels.stats.anova import anova_lm
```

```
In [8]: '''
        `statsmodels.formula.api` 패키지는 통계 모델링과 추정을 위한
        Python 라이브러리 중 하나입니다.
        이 패키지는 선형 회귀 모델링을 위한 `ols()` 함수를 제공합니다.
        여기서는 `ols()` 함수에 대한 자세한 설명을 제공하겠습니다.

        ### `ols()` 함수의 특징:
        1. 선형 회귀 모델링: `ols()` 함수는 Ordinary Least Squares(최소자승법)를
           사용하여 선형 회귀 모델을 적합합니다.
           이는 종속 변수와 하나 이상의 독립 변수 간의 관계를 모델링하는 데 사용됩니다.
        2. 통계적 추론: `ols()` 함수는 회귀 계수의 통계적 유의성을 평가하고,
```

모델의 적합성을 평가하는데 사용되는 통계적 추론을 제공합니다.

3. 편리한 문법: ``statsmodels.formula.api``를 사용하면 R과 유사한 문법을 사용하여 모델을 정의할 수 있습니다. 이는 모델을 간결하고 이해하기 쉽게 만듭니다.

``ols()`` 함수의 인자:

- **formula**: 종속 변수와 독립 변수 간의 관계를 나타내는 모델을 정의하는 문자열입니다. 일반적으로 R 스타일의 수식을 사용합니다.
- **data**: 모델을 적합할 데이터 프레임입니다.
이는 종속 변수와 독립 변수들이 포함된 데이터를 의미합니다.

``ols()`` 함수의 반환값:

``ols()`` 함수는 ``OLS`` 클래스의 인스턴스를 반환합니다.
이 클래스는 다양한 메서드와 속성을 제공하여 회귀 분석 결과를 검사하고 시각화하는 데 사용됩니다.

...

```
import statsmodels.api as sm
```

```
import statsmodels.formula.api as smf
```

데이터 프레임 생성 (예시)

```
data = {
    'x1': [1, 2, 3, 4, 5],
    'x2': [2, 3, 4, 5, 6],
    'y': [5, 7, 9, 11, 13]
}
```

```
df = pd.DataFrame(data)
```

모델 정의 및 적합

```
model = smf.ols(formula='y ~ x1 + x2', data=df)
result = model.fit()
```

회귀 분석 결과 요약

```
print(result.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:          1.000
Model:                  OLS    Adj. R-squared:      1.000
Method:                 Least Squares    F-statistic:    1.709e+30
Date:                   Fri, 03 May 2024    Prob (F-statistic): 9.87e-46
Time:                   09:25:40    Log-Likelihood:    158.99
No. Observations:       5    AIC:              -314.0
Df Residuals:           3    BIC:              -314.8
Df Model:               1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.3333	3.85e-15	3.46e+14	0.000	1.333	1.333
x1	0.3333	2.65e-15	1.26e+14	0.000	0.333	0.333
x2	1.6667	1.25e-15	1.33e+15	0.000	1.667	1.667

```

=====
Omnibus:                nan    Durbin-Watson:          0.011
Prob(Omnibus):           nan    Jarque-Bera (JB):        1.888
Skew:                    1.500    Prob(JB):                 0.389
Kurtosis:                3.250    Cond. No.                 1.77e+16
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The smallest eigenvalue is 4.75e-31. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```

In [9]: '''
`ols()` 함수의 반환값은 `OLS` 클래스의 인스턴스입니다.
이 클래스는 다양한 메서드와 속성을 제공하여
회귀 분석 결과를 검사하고 시각화하는 데 사용됩니다.

### 주요 속성(Attribute):
1. params: 모델의 추정된 계수(회귀 계수)를 나타내는
   파라미터 벡터입니다.
2. summary(): 회귀 분석 결과를 요약한 통계적인 정보를
   제공하는 메서드입니다.
   이를 통해 회귀 분석의 품질, 회귀 계수의
   통계적 유의성 등을 쉽게 파악할 수 있습니다.

```

3. **fittedvalues**: 모델을 통해 추정된 종속 변수의 예측값을 나타내는 벡터입니다.

4. **resid**: 잔차(실제 종속 변수 값과 모델로 예측된 값의 차이)를 나타내는 벡터입니다.

주요 메서드(Method):

1. **fit()**: 모델을 적합시키는 메서드입니다.
이 메서드를 호출하여 회귀 모델을 데이터에 적합시킵니다.
2. **predict()**: 새로운 데이터에 대한 예측을 생성하는 메서드입니다.
이를 통해 새로운 독립 변수 값에 대한 종속 변수의 예측값을 계산할 수 있습니다.
3. **summary()**: 회귀 분석 결과를 요약한 통계적인 정보를 제공하는 메서드입니다. 속성과 동일한 이름을 가진 메서드로, 요약 정보를 텍스트 형식으로 출력합니다.

...

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

# 데이터 프레임 생성 (예시)
data = {
    'x1': [1, 2, 3, 4, 5],
    'x2': [2, 3, 4, 5, 6],
    'y': [5, 7, 9, 11, 13]
}
df = pd.DataFrame(data)

# 모델 정의 및 적합
model = smf.ols(formula='y ~ x1 + x2', data=df)
result = model.fit()

# 회귀 분석 결과 요약
print(result.summary())

# 회귀 계수 출력
print("회귀 계수:", result.params)

# 예측값 출력
print("예측값:", result.fittedvalues)
```

```
# 잔차 출력  
print("잔차:", result.resid)
```

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:          1.000
Model:                  OLS    Adj. R-squared:      1.000
Method:                 Least Squares  F-statistic:    1.709e+30
Date:                   Fri, 03 May 2024  Prob (F-statistic):  9.87e-46
Time:                   09:29:47  Log-Likelihood:   158.99
No. Observations:       5      AIC:             -314.0
Df Residuals:           3      BIC:             -314.8
Df Model:                1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.3333	3.85e-15	3.46e+14	0.000	1.333	1.333
x1	0.3333	2.65e-15	1.26e+14	0.000	0.333	0.333
x2	1.6667	1.25e-15	1.33e+15	0.000	1.667	1.667

```

=====
Omnibus:                nan    Durbin-Watson:          0.011
Prob(Omnibus):          nan    Jarque-Bera (JB):        1.888
Skew:                   1.500    Prob(JB):                0.389
Kurtosis:               3.250    Cond. No.                1.77e+16
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 4.75e-31. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

회귀 계수: Intercept 1.333333

x1 0.333333

x2 1.666667

dtype: float64

예측값: 0 5.0

1 7.0

2 9.0

3 11.0

4 13.0

dtype: float64

잔차: 0 4.440892e-15

1 3.552714e-15

2 3.552714e-15


```
3    3.552714e-15
4    3.552714e-15
dtype: float64
```

```
In [10]: '''
`ols()` 함수는 `statsmodels.formula.api` 패키지에서 제공하는 함수이며,
이는 모델을 정의하고 적합시키기 위한 함수입니다.
반면에 `OLS()` 함수는 `statsmodels.api` 패키지에서 제공하는 함수로,
이는 직접적으로 데이터와 모델을 제어하여 회귀 분석을 수행하기 위한 클래스입니다.

주요 차이점은 다음과 같습니다:

1. 모델 정의 방법:
- `ols()` 함수는 R 스타일의 수식을 사용하여
  모델을 정의합니다.
- `OLS()` 함수는 종속 변수와 독립 변수를 직접적으로 지정하여
  모델을 정의합니다.

2. 사용하는 패키지:
- `ols()` 함수는 `statsmodels.formula.api` 패키지에 포함되어 있습니다.
- `OLS()` 함수는 `statsmodels.api` 패키지에 포함되어 있습니다.

3. 사용하는 방법:
- `ols()` 함수는 데이터프레임과 문자열로 된 모델 수식을 사용하여
  모델을 정의하고 적합시킵니다.
- `OLS()` 함수는 행렬 형태의 독립 변수와 종속 변수를
  직접 입력으로 받아 모델을 정의하고 적합시킵니다.

따라서, 선택은 주어진 상황과 사용자의 선호에 따라 달라집니다.
데이터의 형태와 분석 목적에 따라 더 편리하거나 적합한 방법을 선택할 수 있습니다.
'''

'''
이 예제에서는 다음과 같은 작업을 수행합니다:

1. 먼저 `numpy`를 사용하여 100x2 크기의 랜덤한 독립 변수 행렬
   `X`와 그에 해당하는 종속 변수 벡터 `y`를 생성합니다.
2. 상수항을 추가하기 위해 `sm.add_constant()` 함수를 사용하여
   독립 변수 행렬에 상수 열을 추가합니다.
3. `OLS()` 함수를 사용하여 선형 회귀 모델을 생성합니다.
   이 때 종속 변수 `y`와 독립 변수 행렬 `X`를 입력으로 사용합니다.
```

4. `fit()` 메서드를 호출하여 모델을 적합시킵니다.
이렇게 하면 회귀 분석 결과가 반환됩니다.
5. `summary()` 메서드를 사용하여
회귀 분석 결과를 요약하여 출력합니다.

위 코드를 실행하면 회귀 분석 결과가 요약되어 출력됩니다.
해당 결과를 통해 회귀 계수의 통계적 유의성, 설명력 등을 평가할 수 있습니다.
...

```
# 랜덤한 데이터 생성
np.random.seed(0)
X = np.random.rand(100, 2) # 100x2 크기의 랜덤한 독립 변수 생성
y = np.dot(X, np.array([1.5, 2.0])) + np.random.normal(0, 1, 100) # 종속 변수 생성

# 상수항 추가
X = sm.add_constant(X)

# OLS 모델 생성 및 적합
model = sm.OLS(y, X)
result = model.fit()

# 결과 출력
print(result.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:                0.323
Model:                  OLS    Adj. R-squared:           0.309
Method:                 Least Squares    F-statistic:        23.13
Date:                   Fri, 03 May 2024    Prob (F-statistic):  6.11e-09
Time:                   09:33:24    Log-Likelihood:     -133.53
No. Observations:       100    AIC:                273.1
Df Residuals:           97    BIC:                280.9
Df Model:                2
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.1145	0.259	0.442	0.659	-0.400	0.629
x1	1.0737	0.332	3.230	0.002	0.414	1.733
x2	2.0296	0.328	6.193	0.000	1.379	2.680

```

=====
Omnibus:                0.410    Durbin-Watson:        2.045
Prob(Omnibus):          0.815    Jarque-Bera (JB):      0.502
Skew:                   0.144    Prob(JB):              0.778
Kurtosis:               2.807    Cond. No.              5.58
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [5]: '''
`anova_lm()` 함수는 Python의 `statsmodels` 라이브러리에서
제공되는 함수 중 하나입니다.
이 함수는 분산분석(ANOVA, Analysis of Variance)을
수행하는 데 사용됩니다.
ANOVA는 그룹 간의 평균 차이를 비교하여 변수 간의 관계를
검정하는 통계적인 방법입니다.
주로 세 개 이상의 그룹 간에 평균 차이가 있는지 여부를 판단하는 데 사용됩니다.

`anova_lm()` 함수는 일반적으로 선형 회귀 모델 또는
다중 선형 회귀 모델의 ANOVA 분석 결과를 제공합니다.

1. model: 선형 회귀 모델 객체 또는 다중 선형 회귀 모델 객체.
   ANOVA 분석을 수행할 모델입니다.
'''

```

2. **type**: 분산 분석 테이블의 종류를 지정하는 문자열.
기본값은 "I"로, 이것은 요인 변수(독립 변수)가 더해진 상태를 나타냅니다.
"II"는 요인 변수의 평균이 빠진 상태를 나타냅니다.
3. **robust**: 논리 값(기본값은 False). True로 설정하면
로버스트 표준 오차가 사용됩니다.
이는 회귀 모델이 잘못된 분산 구조를 가지고 있는 경우에 유용합니다.
4. **covar_type**: 더 복잡한 공분산 구조를 고려할 때
사용되는 문자열. 기본값은 'nonrobust'입니다.

이 함수는 F-통계량, p-값 및 다양한 요인의 자유도를 포함한 ANOVA 테이블을 반환합니다.
이를 통해 모델의 설명력을 평가하거나 그룹 간 평균 차이의 통계적 유의성을 평가할 수 있습니다.
ANOVA는 그룹 간의 평균 차이를 비교할 때 사용되며,
그룹 간에 통계적으로 유의미한 차이가 있는지 여부를 결정하는 데 유용합니다.

```
'''
1. `ols('value ~ group', data=data1).fit()`:
이 코드는 Ordinary Least Squares(OLS) 방법을 사용하여
선형 회귀 모델을 적합시킵니다.
여기서 'value'는 종속 변수이고 'group'은 독립 변수입니다.
`data=data1`은 분석에 사용할 데이터프레임을 지정합니다.
즉, 'data1'이라는 데이터프레임에서 'value'와 'group' 열을 사용하여
선형 회귀 모델을 적합시킵니다.
2. `.fit()`: `ols()` 함수로 정의된 모델을 데이터에 맞추니다.
이렇게 하면 선형 회귀 모델이 생성되고 해당 데이터에 최적화됩니다.
`fit()` 메서드의 결과로는 모델 객체가 반환됩니다.
여기서는 이 모델 객체가 변수 `anova1`에 할당됩니다.
3. `anova_lm(anova1)`: 이 코드는 ANOVA(Analysis of Variance)
분석을 수행합니다.
`anova_lm()` 함수는 전달된 선형 회귀 모델에 대한
ANOVA 결과를 계산합니다.
여기서는 `anova1`에 저장된 선형 회귀 모델을 사용하여
ANOVA를 수행하고, ANOVA 테이블을 반환합니다.
결과적으로, 이 코드는 'value'와 'group' 사이의 관계를 검정하기 위해
선형 회귀 모델을 적합하고, 이 모델에 대한 ANOVA를 수행하여
그룹 간의 평균 차이를 평가합니다.
'''
```

```
anova1 = ols('value ~ group', data = data1).fit()
anova_lm(anova1)
```

```
Out[5]:
```

	df	sum_sq	mean_sq	F	PR(>F)
group	2.0	144.933333	72.466667	28.233766	0.000029
Residual	12.0	30.800000	2.566667	NaN	NaN

```
In [6]: '''
이 코드는 Python의 `scipy.stats` 모듈에서 제공되는 함수 중
하나인 `f.ppf()`를 사용합니다.
이 함수는 F 분포의 퍼센트 포인트 함수의
역함수(inverse cumulative distribution function)로,
주어진 확률값에 해당하는 F 분포의 값을 계산합니다.

여기서 각각의 인자는 다음과 같습니다:

- `0.95`: 원하는 확률 값이며, 여기서는 95% 신뢰 수준을 나타냅니다.
  이 확률 값은 F 분포의 우측 꼬리 영역의 비율을 의미합니다.
- `dfn=2`: 분자 자유도입니다. 일반적으로 그룹 수에 해당하는 값입니다.
- `dfd=12`: 분모 자유도입니다.
  일반적으로 데이터 포인트 수에서 그룹 수를 뺀 값입니다.

이 코드는 95%의 신뢰 수준에서, 분자 자유도가 2이고 분모 자유도가 12인
F 분포의 퍼센트 포인트 값을 계산합니다.
이 값은 주어진 자유도와 신뢰 수준에서 F 분포의 임계값을 나타냅니다.
일반적으로 이 임계값은 통계적 가설 검정에서 유의수준을 설정하거나,
신뢰구간을 계산할 때 사용됩니다.
'''

stats.f.ppf(0.95, dfn=2, dfd=12)
```

```
Out[6]: 3.8852938346523933
```

등분산 검정

```
In [1]: '''
코드는 세 개의 그룹에 대해 Bartlett의 등분산성 검정을 수행합니다.
```

결과로는 검정 통계량과 p-값이 출력됩니다.
 p-값이 일반적으로 선택한 유의수준보다 작으면
 등분산성 가정을 기각하고, 그렇지 않으면 등분산성을 채택합니다.
 '''

```
from scipy.stats import bartlett
```

예시 데이터

```
group1 = [10, 12, 14, 16, 18]
```

```
group2 = [9, 11, 13, 15, 17]
```

```
group3 = [8, 10, 12, 14, 16]
```

등분산성 검정 수행

```
statistic, pvalue = bartlett(group1, group2, group3)
```

결과 출력

```
print("Statistic:", statistic)
```

```
print("P-value:", pvalue)
```

Statistic: 0.0

P-value: 1.0

```
In [7]: data1_A = data1[data1['group']=='A']['value']
        data1_B = data1[data1['group']=='B']['value']
        data1_C = data1[data1['group']=='C']['value']
```

```
In [8]: '''
`bartlett()` 함수는 Python의 `scipy.stats` 모듈에서
제공되는 함수 중 하나입니다.
이 함수는 Bartlett의 등분산성 검정(Bartlett's test for equal variances)을
수행하는 데 사용됩니다. 등분산성 검정은 두 개 이상의 그룹이나
처리의 모집단에서 분산이 동일한지 여부를 평가하는 통계적인 방법입니다.
```

특징:

- Bartlett의 등분산성 검정은
각 그룹의 표본 크기가 동일한 경우에 사용됩니다.
- 등분산성 가정은 일부 통계적 검정(예: 분산분석)에 필요한
가정 중 하나입니다. 등분산성 가정이 만족되지 않으면
검정 결과가 부정확 할 수 있습니다.

인자:

```
- `*args`: 검정을 수행할 그룹의 데이터를 전달합니다.  
    여기서 각 그룹은 리스트, 배열 또는 Pandas Series와 같은  
    반복 가능한 형식이어야 합니다.  
  
    반환값:  
- `statistic`: 검정 통계량(Bartlett의 통계량).  
- `pvalue`: 검정 결과에 대한 p-값.  
...  
  
stats.bartlett(data1_A, data1_B, data1_C)
```

```
Out[8]: BartlettResult(statistic=2.8944987583039246, pvalue=0.2352163902021605)
```

Twoway ANOVA

```
In [9]: data2 = pd.read_csv("marketing.csv")  
data2
```

Out[9]:

	media	strategy	sales
0	Television	Convenience	491
1	Television	Convenience	712
2	Television	Convenience	558
3	Television	Convenience	447
4	Television	Convenience	479
5	Television	Convenience	624
6	Television	Convenience	546
7	Television	Convenience	444
8	Television	Convenience	582
9	Television	Convenience	672
10	Newspaper	Convenience	464
11	Newspaper	Convenience	559
12	Newspaper	Convenience	759
13	Newspaper	Convenience	557
14	Newspaper	Convenience	528
15	Newspaper	Convenience	670
16	Newspaper	Convenience	534
17	Newspaper	Convenience	657
18	Newspaper	Convenience	557
19	Newspaper	Convenience	474
20	Television	Quality	677
21	Television	Quality	627

	media	strategy	sales
22	Television	Quality	590
23	Television	Quality	632
24	Television	Quality	683
25	Television	Quality	760
26	Television	Quality	690
27	Television	Quality	548
28	Television	Quality	579
29	Television	Quality	644
30	Newspaper	Quality	689
31	Newspaper	Quality	650
32	Newspaper	Quality	704
33	Newspaper	Quality	652
34	Newspaper	Quality	576
35	Newspaper	Quality	836
36	Newspaper	Quality	628
37	Newspaper	Quality	798
38	Newspaper	Quality	497
39	Newspaper	Quality	841
40	Television	Price	575
41	Television	Price	614
42	Television	Price	706
43	Television	Price	484

	media	strategy	sales
44	Television	Price	478
45	Television	Price	650
46	Television	Price	583
47	Television	Price	536
48	Television	Price	579
49	Television	Price	795
50	Newspaper	Price	803
51	Newspaper	Price	584
52	Newspaper	Price	525
53	Newspaper	Price	498
54	Newspaper	Price	812
55	Newspaper	Price	565
56	Newspaper	Price	708
57	Newspaper	Price	546
58	Newspaper	Price	616
59	Newspaper	Price	587

In [10]:

```
'''
이 코드는 다중 선형 회귀 모델을 사용하여
ANOVA(Analysis of Variance) 분석을 수행합니다.
여기서는 Python의 `statsmodels` 라이브러리를 사용합니다.

1. `ols('sales ~ strategy + media + strategy:media',
data=data2).fit()`: 이 코드는 선형 회귀 모델을 생성하고
적합시킵니다. 모델식은 'sales ~ strategy + media + strategy:media'로,
종속 변수 'sales'와 'strategy', 'media', 그리고
```

```

두 변수의 상호작용 항목인 'strategy:media'가 포함됩니다.
'data=data2'는 이 모델을 적합시킬 데이터를 지정합니다.
2. .fit(): 이 메서드는 데이터에 모델을 맞춥니다.
즉, 모델을 데이터에 최적화시키고 최적의 회귀계수를 찾습니다.
이 작업을 마치면 회귀 모델 객체가 반환됩니다.
3. anova_lm(anova2): 이 코드는 ANOVA(Analysis of Variance)
테이블을 생성합니다. anova_lm() 함수는 전달된 선형 회귀 모델에 대한
ANOVA 결과를 계산합니다. 여기서는 이전에 적합한 선형 회귀 모델인
anova2를 사용하여 ANOVA를 수행하고, ANOVA 테이블을 반환합니다.
결과적으로, 이 코드는 'sales'를 종속 변수로 하는
다중 선형 회귀 모델을 적합하고,
이 모델에 대한 ANOVA를 수행하여
'strategy', 'media', 그리고 두 변수의 상호작용에 대한
통계적 유의성을 평가합니다.
'''

anova2 = ols('sales ~ strategy + media + strategy:media',
              data = data2).fit()
anova_lm(anova2)

```

Out[10]:

	df	sum_sq	mean_sq	F	PR(>F)
strategy	2.0	98838.633333	49419.316667	5.325180	0.007748
media	1.0	13172.016667	13172.016667	1.419351	0.238720
strategy:media	2.0	1609.633333	804.816667	0.086723	0.917058
Residual	54.0	501136.700000	9280.309259	NaN	NaN

In [4]:

```

'''
interaction_plot() 함수는 Python의
statsmodels.graphics.factorplots 모듈에서 제공됩니다.
이 함수는 상호작용 효과를 시각화하는 데 사용됩니다.
주로 두 개의 요인(factor)이 존재할 때
이들 간의 상호작용을 이해하고 시각적으로 표현하는 데 유용합니다.

특징:
- 두 개의 요인(factor) 간의 상호작용 효과를 그래프로 표현합니다.
- 상호작용 효과를 시각적으로 이해하고,

```

각 요인이 종속 변수에 미치는 영향을 확인할 수 있습니다.

인자:

- ``x``: x축에 표시될 요인(factor)의 데이터.
일반적으로 범주형 변수의 값을 가집니다.
- ``trace``: 플롯할 그룹 간의 상호작용을 표시하는 데 사용됩니다.
- ``response``: 종속 변수의 데이터.
- ``colors``: 각 그룹에 대한 색상을 지정합니다.
- ``markers``: 각 그룹에 대한 마커 스타일을 지정합니다.
- ``markersize``: 마커의 크기를 지정합니다.
- ``ax``: 그래프를 그릴 Matplotlib 축을 지정합니다.

반환값:

- ``fig``: 생성된 그림(Figure) 객체
- ``ax``: 생성된 축(Axes) 객체

위 코드는 두 개의 요인(factor)에 대한 상호작용 효과를 시각화하는 방법을 보여줍니다.

이를 통해 각 요인이 종속 변수에 미치는 영향과 그들 간의 상호작용을 쉽게 이해할 수 있습니다.
'''

```
from statsmodels.graphics.factorplots import interaction_plot
import matplotlib.pyplot as plt
```

예시 데이터

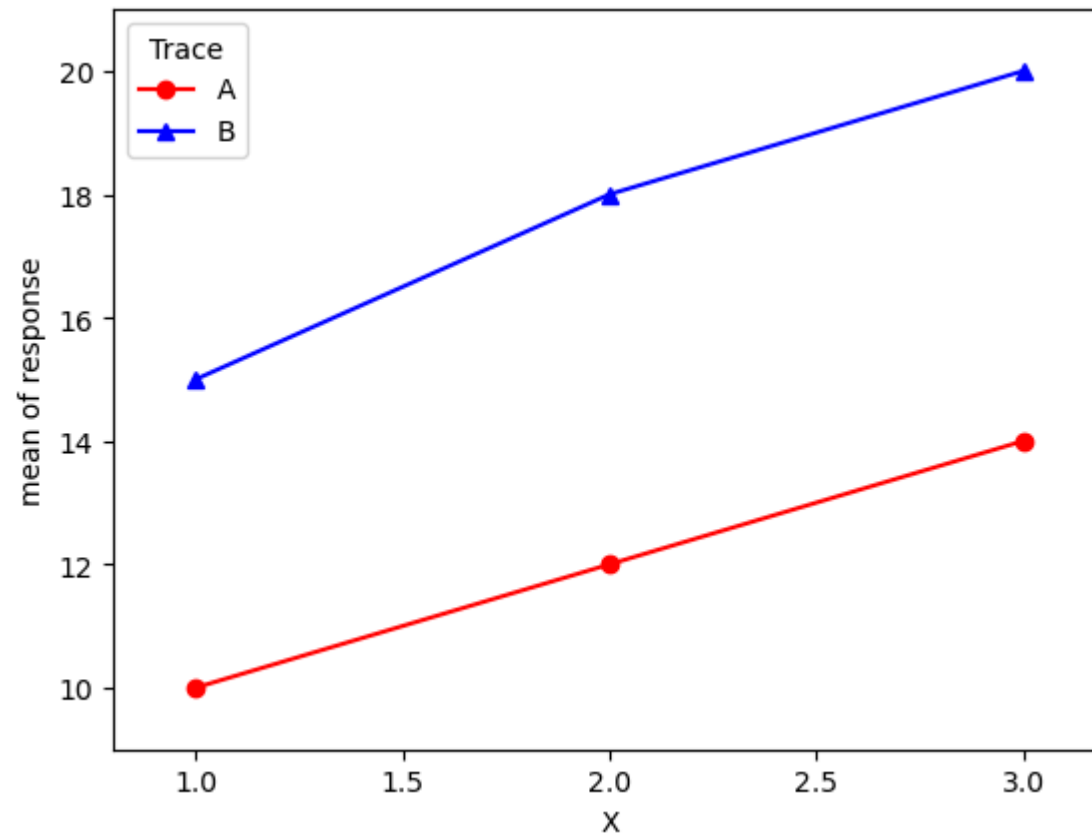
```
factor1 = [1, 1, 2, 2, 3, 3]
factor2 = ['A', 'B', 'A', 'B', 'A', 'B']
response = [10, 15, 12, 18, 14, 20]
```

상호작용 플롯 그리기

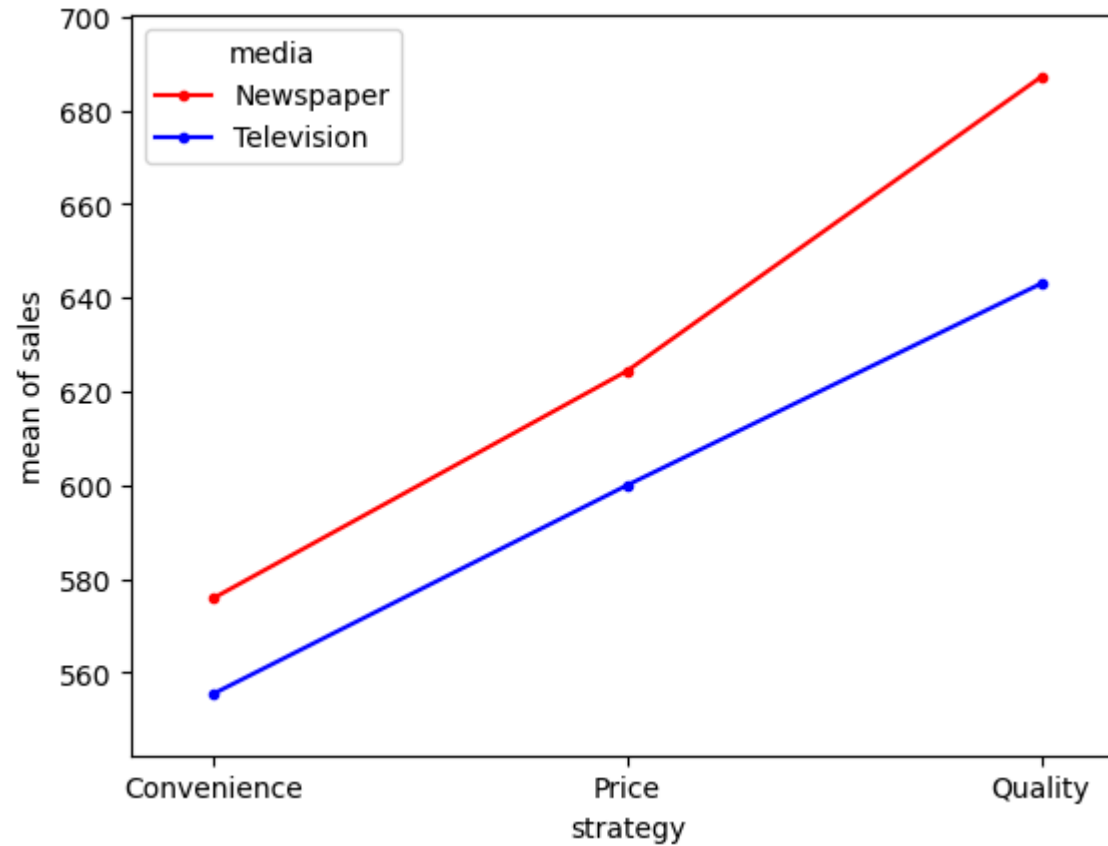
```
fig, ax = plt.subplots()
interaction_plot(x=factor1, trace=factor2,
                response=response, colors=['red', 'blue'],
                markers=['o', '^'], ax=ax)
```

그래프 표시

```
plt.show()
```



```
In [11]: from statsmodels.graphics.api import interaction_plot
interaction_plot(data2["strategy"], data2["media"], data2["sales"],
                colors=["red", "blue"])
plt.show()
```



범주형 자료분석 예제

적합성 검정

```
In [12]: Obs = np.array([285, 66, 55, 44])  
Pr = np.array([0.68, 0.13, 0.11, 0.08])  
n = Obs.sum(); E = n*Pr; df = len(Obs)-1
```

```
In [13]: print(" Observed :", Obs, "\n",  
              "Proportion :", Pr, "\n",
```

```
"Expected :",E)
```

```
Observed : [285  66  55  44]
Proportion : [0.68 0.13 0.11 0.08]
Expected : [306.   58.5  49.5  36. ]
```

```
In [5]: '''
`chisquare()` 함수는 Python의 `scipy.stats` 모듈에서
제공되는 함수 중 하나입니다.
이 함수는 카이제곱 검정(Chi-squared test)을
수행하는 데 사용됩니다.
카이제곱 검정은 두 범주형 변수 간의 관련성을 평가하거나,
하나의 범주형 변수의 분포가 예상되는 분포와
일치하는지 여부를 확인하는 데 사용됩니다.

특징:
- 카이제곱 검정은 두 범주형 변수 간의
  독립성을 검정하는 데 주로 사용됩니다.
- 기대되는 빈도와 실제 관찰된 빈도 간의 차이를 측정하여
  통계적 유의성을 확인합니다.
- 주로 분할표(Contingency Table) 데이터를
  기반으로 검정을 수행합니다.

인자:
- `f_obs`: 관찰된 빈도를 나타내는 배열 또는 리스트입니다.
  이는 각 범주형 변수의 관찰된 빈도를 포함하는
  분할표(Contingency Table)의 형태를 가집니다.
- `f_exp`: 기대되는 빈도를 나타내는 배열 또는 리스트입니다.
  이는 각 범주형 변수의 예상되는 빈도를 포함하는
  분할표(Contingency Table)의 형태를 가집니다.
  기본값은 None으로, 기대되는 빈도를 관찰된 빈도에서 추정합니다.
- `ddof`: 자유도 조정 값입니다. 기본값은 0이며,
  이는 자유도를 조정하지 않음을 의미합니다.

반환값:
- `statistic`: 카이제곱 검정 통계량입니다.
- `pvalue`: 검정 결과에 대한 p-값입니다.
'''

'''
코드는 두 개의 범주형 변수 간의 독립성을 검정하기 위해
```

카이제곱 검정을 수행하는 방법을 보여줍니다.
 결과로는 검정 통계량과 p-값이 출력됩니다.
 p-값이 일반적으로 선택한 유의수준보다 작으면 귀무가설을 기각하고,
 두 변수 간에는 통계적으로 유의한 관계가 있다고 할 수 있습니다.
 ...

```
from scipy.stats import chisquare
```

```
# 예시 데이터
```

```
observed = [30, 20, 15]
```

```
expected = [25, 25, 15]
```

```
# 카이제곱 검정 수행
```

```
statistic, pvalue = chisquare(f_obs=observed, f_exp=expected)
```

```
# 결과 출력
```

```
print("Chi-square Statistic:", statistic)
```

```
print("P-value:", pvalue)
```

```
Chi-square Statistic: 2.0
```

```
P-value: 0.36787944117144245
```

```
In [14]: chi2, p = stats.chisquare(Obs ,E)
```

```
In [15]: print (" Chi-Squared :", round(chi2, 4), "\n",
              "df :", df, "\n",
              "P-Value :", round(p, 4), "\n",
              "critical value :", round(stats.chi2.ppf(0.95, df), 4))
```

```
Chi-Squared : 4.7916
```

```
df : 3
```

```
P-Value : 0.1877
```

```
critical value : 7.8147
```

독립성 검정

```
In [13]: defective = np.array([[11,25,27],[15,31,28],[44,24,52],[10,17,16]])
          column_names = ['Line1', 'Line2', 'Line3']
          row_names     = ['상판', '다리', '서랍', '도색']
          table = pd.DataFrame(defective, columns=column_names,
```



```
index=row_names)
table
```

Out[13]:

	Line1	Line2	Line3
상판	11	25	27
다리	15	31	28
서랍	44	24	52
도색	10	17	16

In [14]:

```
'''
`chi2_contingency()` 함수는 카이제곱 독립성 검정을 수행하는
파이썬의 SciPy 라이브러리에서 제공되는 함수입니다.
이 함수를 사용하여 두 개 이상의 범주형 변수 간의 독립성을 검정할 수 있습니다.

사용법:
from scipy.stats import chi2_contingency

chi2_stat, p_val, dof, expected =
    chi2_contingency(observed, correction=True)

인자:
- `observed` (배열 또는 배열의 리스트): 관측된 빈도를 포함하는 배열
  또는 배열의 리스트입니다. 만약 두 개의 변수가 있다면, 이는 2차원 배열이며
  각 행은 첫 번째 변수의 각 범주에서 관측된 빈도를 나타내고,
  각 열은 두 번째 변수의 각 범주에서 관측된 빈도를 나타냅니다.
- `correction` (부울, 선택적): 기본값은 True입니다.
  이것은 Yates' continuity correction을 사용할지 여부를 지정합니다.
  보통은 True로 설정하며, 표본 크기가 작은 경우 특히 사용됩니다.

리턴값:
- `chi2_stat` (부동 소수점): 카이제곱 통계량입니다.
  이 값이 클수록 변수 간의 연관성이 적다는 것을 의미합니다.
- `p_val` (부동 소수점): p-value 값으로, 변수 간의
  독립성을 검정하는 데 사용됩니다. 유의 수준과 비교하여 변수 간의 관계가
  통계적으로 유의한지 확인할 수 있습니다.
- `dof` (정수): 자유도(degree of freedom)입니다.
```

- ``expected`` (배열 또는 배열의 리스트): 기대 빈도를 포함하는 배열 또는 배열의 리스트입니다. 만약 두 개의 변수가 있다면, 이는 2차원 배열이며 각 행은 첫 번째 변수의 각 범주에서 예상되는 빈도를 나타내고, 각 열은 두 번째 변수의 각 범주에서 예상되는 빈도를 나타냅니다.

특징:

- 이 함수는 관측된 빈도를 사용하여 기대 빈도를 계산하고, 이를 기반으로 카이제곱 통계량과 **p-value**를 계산합니다.
- 카이제곱 독립성 검정은 두 개 이상의 범주형 변수 간의 관계를 이해하고자 할 때 사용됩니다. 만약 **p-value**가 유의 수준보다 작다면, 변수들 간의 관계가 통계적으로 유의하다고 볼 수 있습니다.
- 이 함수는 기본적으로 **Pearson** 카이제곱 검정을 수행하지만, **Yates' continuity correction**을 적용할 수도 있습니다.
- 일반적으로, 변수 간의 관계를 분석하기 위해 표본 크기가 충분히 큰 경우에 사용됩니다.

이 함수를 사용하여 두 개 이상의 범주형 변수 간의 관계를 통계적으로 검정할 수 있습니다.

```
'''
```

```
chi22, p2, dof, expected = stats.chi2_contingency(defective)
print (" Pearson's Chi-squared test","\n",
      "Chi-Squared :",round(chi22,4),"\n",
      "df :",dof,"\n",
      "P-Value :", round(p2, 4),"\n",
      "critical value :", round(stats.chi2.ppf(0.95, dof),4))
```

```
Pearson's Chi-squared test
Chi-Squared : 17.8474
df : 6
P-Value : 0.0066
critical value : 12.5916
```

```
In [15]: expected
```

```
Out[15]: array([[16.8       , 20.37       , 25.83       ],
                [19.73333333, 23.92666667, 30.34       ],
                [32.        , 38.8        , 49.2        ],
                [11.46666667, 13.90333333, 17.63       ]])
```

비모수 검정 예시

Mann Whitney U 검정

```
In [20]: '''
`mannwhitneyu()` 함수는 두 개의 독립적인 표본 간의
비모수적인 Mann-Whitney U 검정(또는 Wilcoxon 순위 합 검정)을
수행하는 파이썬의 SciPy 라이브러리에서 제공되는 함수입니다.

사용법:
from scipy.stats import mannwhitneyu
statistic, p_value = mannwhitneyu(x, y, use_continuity=True,
                                  alternative=None)

인자:
- `x`, `y` (배열 또는 배열의 리스트): 비교하려는 두 개의 표본입니다.
  이는 1차원 배열이어야 합니다.

- `use_continuity` (부울, 선택적): 기본값은 True입니다.
  이것은 연속성 보정(continuity correction)을 사용할지 여부를 지정합니다.
- `alternative` (문자열, 선택적): 기본값은 None입니다.
  다음 중 하나를 선택할 수 있습니다: 'two-sided', 'less', 'greater'.
  이는 대립가설을 설정합니다. 'two-sided'는 양측검정, 'less'는 좌측검정,
  'greater'는 우측검정입니다.

리턴값:
- `statistic` (부동 소수점): Mann-Whitney U 통계량입니다.
  두 표본 간의 차이를 나타내는 값입니다.
- `p_value` (부동 소수점): 검정의 유의성을 나타내는 p-value 값입니다.
  유의 수준과 비교하여 표본 간의 차이가
  통계적으로 유의한지 확인할 수 있습니다.

특징:
- Mann-Whitney U 검정은 두 개의 독립적인 표본이
  서로 동일한 분포를 따르는지 여부를 검정합니다.
- 이 검정은 두 표본 모집단이 정규분포를 따르지 않을 때 사용됩니다.
'''
```

- 데이터가 순위로 변환되어 비교되므로, 표본 크기나 분포에 대한 가정이 필요하지 않습니다.
- 유의수준에 따라 **p-value**를 해석하여 귀무가설을 기각하거나 기각하지 않습니다. 일반적으로, 유의 수준을 **0.05**로 선택합니다.
- 이 검정은 두 표본 간의 위치 차이를 비교하며, 두 표본이 크기나 분포 면에서 다르다는 것을 나타냅니다.

따라서, ``mannwhitneyu()`` 함수는 비모수적인 방법을 사용하여 두 개의 독립적인 표본 간의 차이를 검정하는 데 유용합니다.

```
'''
```

```
x = [70, 60, 45, 90, 95, 80, 85]
y = [100, 95, 55, 65, 75, 75]
stats.mannwhitneyu(x, y)
```

Out[20]: MannwhitneyuResult(statistic=19.5, pvalue=0.8860917695085568)

Wilcoxon Signed-rank 검정

In [21]: `d = [1.5, 1.5, -3, -4, -5, -6, 7, 8, 9]`

In [22]: `'''`

``wilcoxon()`` 함수는 두 개의 관련된 표본 간의 비모수적인 Wilcoxon 부호 순위 합 검정을 수행하는 파이썬의 SciPy 라이브러리에서 제공되는 함수입니다.

사용법:

```
from scipy.stats import wilcoxon
statistic, p_value = wilcoxon(x, y, zero_method='wilcox',
                             correction=False, alternative='two-sided')
```

인자:

- ``x`, `y`` (배열 또는 배열의 리스트): 비교하려는 두 개의 관련된 표본입니다. 이는 1차원 배열이어야 합니다.
- ``zero_method`` (문자열, 선택적): 기본값은 `'wilcox'`입니다. 이것은 0에 대한 처리 방법을 지정합니다. `'wilcox'`는 Wilcoxon 부호 순위 합 테스트의 표준 방법을 사용하며, `'pratt'`는 Pratt 방법을 사용합니다.

- ``correction`` (부울, 선택적): 기본값은 `False`입니다.
이것은 연속성 보정(`continuity correction`)을 사용할지 여부를 지정합니다.
- ``alternative`` (문자열, 선택적): 기본값은 `'two-sided'`입니다.
다음 중 하나를 선택할 수 있습니다: `'two-sided'`, `'less'`, `'greater'`.
이는 대립가설을 설정합니다. `'two-sided'`는 양측검정, `'less'`는 좌측검정, `'greater'`는 우측검정입니다.

리턴값:

- ``statistic`` (부동 소수점): Wilcoxon 부호 순위 합 통계량입니다.
두 표본 간의 차이를 나타내는 값입니다.
- ``p_value`` (부동 소수점): 검정의 유의성을 나타내는 `p-value` 값입니다.
유의 수준과 비교하여 표본 간의 차이가 통계적으로 유의한지 확인할 수 있습니다.

특징:

- Wilcoxon 부호 순위 합 검정은 두 개의 관련된 표본 간의 차이를 비교합니다.
관련된 표본이란 동일한 개체나 집단에서 추출된 두 표본을 의미합니다.
- 이 검정은 두 표본 모집단이 정규분포를 따르지 않을 때 사용됩니다.
- 데이터가 순위로 변환되어 비교되므로,
표본 크기나 분포에 대한 가정이 필요하지 않습니다.
- 유의수준에 따라 `p-value`를 해석하여 귀무가설을 기각하거나 기각하지 않습니다. 일반적으로, 유의 수준을 `0.05`로 선택합니다.
- 이 검정은 관련된 표본 간의 위치 차이를 비교하며,
두 표본이 크기나 분포 면에서 다르다는 것을 나타냅니다.

따라서, ``wilcoxon()`` 함수는 비모수적인 방법을 사용하여 두 개의 관련된 표본 간의 차이를 검정하는 데 유용합니다.
'''

```
stats.wilcoxon(d)
```

```
Out[22]: WilcoxonResult(statistic=18.0, pvalue=0.65234375)
```

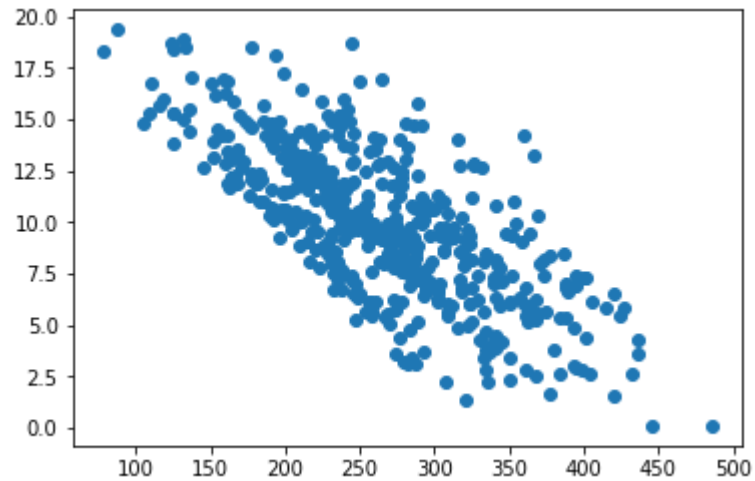
상관계수 검정

```
In [23]: used_car = pd.read_csv("used_car.csv")
used_car.head()
```

```
x=used_car['price']
y=used_car['odometer']
```

산점도를 그려 선형관계 파악하기

```
In [24]: plt.scatter(x, y)
plt.show()
```



Pearson correlation coefficient

```
In [ ]: '''
`pearsonr()` 함수는 두 변수 간의 피어슨 상관계수를 계산하는
파이썬의 SciPy 라이브러리에서 제공되는 함수입니다.
```

사용법:

```
from scipy.stats import pearsonr
correlation_coefficient, p_value = pearsonr(x, y)
```

인자:

```
- `x`, `y` (배열 또는 배열의 리스트): 상관관계를 계산할
  두 변수의 데이터입니다. 이는 1차원 배열이어야 합니다.
```

리턴값:

```

- `correlation_coefficient` (부동 소수점):
    두 변수 간의 피어슨 상관계수 값입니다.
    -1에서 1까지의 범위를 가지며, 1에 가까울수록
    양의 선형 상관관계를 나타내고, -1에 가까울수록
    음의 선형 상관관계를 나타냅니다.
    0은 상관관계가 없음을 나타냅니다.

- `p_value` (부동 소수점): 상관계수의 유의성을 나타내는
    p-value 값입니다. 유의 수준과 비교하여 상관관계가
    통계적으로 유의한지 확인할 수 있습니다.

특징:
- `pearsonr()` 함수는 두 변수 간의 선형 상관관계를 계산합니다.
- 상관계수는 -1부터 1까지의 범위를 가지며, 1에 가까울수록
    양의 선형 상관관계를, -1에 가까울수록
    음의 선형 상관관계를 나타냅니다.
    0은 선형 관계가 없음을 나타냅니다.
- 이 함수는 두 변수가 정규 분포를 따르고 있을 때 가장 잘 작동합니다.
- 피어슨 상관계수는 이상치에 민감할 수 있으므로,
    데이터에 이상치가 있는 경우 스피어만 상관계수나
    켄달 상관계수를 고려하는 것이 좋을 수 있습니다.
- 상관계수의 유의성을 나타내는 p-value 값은
    두 변수 간의 상관관계가 통계적으로 유의한지
    여부를 판단하는 데 사용됩니다.

따라서, `pearsonr()` 함수는 두 변수 간의
선형 상관관계를 계산하고 그 유의성을 평가하는 데 사용됩니다.
'''

stats.pearsonr(x, y)

```

Spearman correlation coefficient

```

In [26]: '''
`spearmanr()` 함수는 두 변수 간의 스피어만
순위 상관계수를 계산하는 파이썬의
SciPy 라이브러리에서 제공되는 함수입니다.

사용법:

```

```
from scipy.stats import spearmanr
correlation_coefficient, p_value = spearmanr(x, y)
```

인자:

- ``x``, ``y`` (배열 또는 배열의 리스트): 상관관계를 계산할 두 변수의 데이터입니다. 이는 1차원 배열이어야 합니다.

리턴값:

- ``correlation_coefficient`` (부동 소수점): 두 변수 간의 스피어만 순위 상관관계수 값입니다. -1에서 1까지의 범위를 가지며, 1에 가까울수록 양의 순위 상관관계를 나타내고, -1에 가까울수록 음의 순위 상관관계를 나타냅니다. 0은 순위 상관관계가 없음을 나타냅니다.
- ``p_value`` (부동 소수점): 상관관계수의 유의성을 나타내는 p-value 값입니다. 유의 수준과 비교하여 상관관계가 통계적으로 유의한지 확인할 수 있습니다.

특징:

- ``spearmanr()`` 함수는 두 변수 간의 순위 상관계수를 계산합니다.
- 스피어만 상관계수는 변수 간의 선형적인 관계가 아니라 순위에 기반한 관계를 측정합니다. 따라서, 변수 간의 비선형적인 관계도 탐지할 수 있습니다.
- 이 함수는 변수가 정규 분포를 따르지 않을 때 사용됩니다.
- 스피어만 상관계수는 이상치에 비교적 덜 민감하며, 특히 순위를 사용하여 계산하기 때문에 데이터의 분포에 덜 영향을 받습니다.
- 상관관계수의 유의성을 나타내는 p-value 값은 두 변수 간의 상관관계가 통계적으로 유의한지 여부를 판단하는 데 사용됩니다.

따라서, ``spearmanr()`` 함수는 두 변수 간의 순위 상관계수를 계산하여 변수 간의 순서에 따른 관계를 평가하는 데 사용됩니다.

```
...
```

```
stats.spearmanr(x, y)
```

Out[26]: SpearmanrResult(correlation=-0.7177799550205398, pvalue=2.5095391491547877e-80)

Kendall's tau coefficient


```
In [27]: x = [1, 2, 3, 4, 5]
        y = [3, 4, 1, 2, 5]
```

```
In [28]: stats.spearmanr(x, y)
```

```
Out[28]: SpearmanrResult(correlation=0.19999999999999998, pvalue=0.747060078104662)
```

```
In [29]: '''
`kendalltau()` 함수는 두 변수 간의 켄달의
순위 상관계수를 계산하는 파이썬의
SciPy 라이브러리에서 제공되는 함수입니다.

사용법:
from scipy.stats import kendalltau
correlation_coefficient, p_value = kendalltau(x, y)

인자:
- `x`, `y` (배열 또는 배열의 리스트): 상관관계를 계산할
  두 변수의 데이터입니다. 이는 1차원 배열이어야 합니다.

리턴값:
- `correlation_coefficient` (부동 소수점): 두 변수 간의 켄달의
  순위 상관계수 값입니다. -1에서 1까지의 범위를 가지며,
  1에 가까울수록 양의 순위 상관관계를 나타내고,
  -1에 가까울수록 음의 순위 상관관계를 나타냅니다.
  0은 순위 상관관계가 없음을 나타냅니다.
- `p_value` (부동 소수점): 상관계수의 유의성을 나타내는
  p-value 값입니다. 유의 수준과 비교하여 상관관계가
  통계적으로 유의한지 확인할 수 있습니다.

특징:
- `kendalltau()` 함수는 두 변수 간의
  켄달의 순위 상관계수를 계산합니다.
- 켄달의 상관계수는 변수 간의 선형적인 관계가 아니라
  순위에 기반한 관계를 측정합니다.
  따라서, 변수 간의 비선형적인 관계도 탐지할 수 있습니다.
- 이 함수는 변수가 정규 분포를 따르지 않을 때 사용됩니다.
- 켄달의 상관계수는 이상치에 비교적 덜 민감하며,
  변수 간의 순서에 따른 관계를 평가합니다.
```

- 상관계수의 유의성을 나타내는 **p-value** 값은
두 변수 간의 상관관계가 통계적으로 유의한지
여부를 판단하는 데 사용됩니다.

따라서, ``kendalltau()`` 함수는 두 변수 간의
순위 상관계수를 계산하여 변수 간의 순서에 따른 관계를
평가하는 데 사용됩니다.
...

```
stats.kendalltau(x, y)
```

Out[29]: KendalltauResult(correlation=0.19999999999999998, pvalue=0.8166666666666667)

In []: